

SkillNet-S1 Demo 说明文档

能力总览 · 业务价值 · 真实产品接入指南

版本 v0.4-pre (可信度重构 + 策略层闭环演示) · 文档日期 2026-09-24 · 对应仓库 skillnet-demo (commit f3af611)

阅读对象: 技术负责人 / Agent 系统工程师 / 准备把 SkillNet 接入业务产品的开发者

一句话定位: SkillNet-S1 是给 Agent 产品 (如立理 S1) 做的「技能运维层」原型——把 Agent 的能力以「技能」为单位资产化, 提供检索路由、编排执行、效果度量与进化治理的完整闭环。它要回答的问题是: **Agent 的技能从哪来、怎么找对、用了有没有效、如何持续变好。**

一、Demo 当前能做什么

Demo 是一个可一键运行的完整系统: FastAPI 后端 (默认 `http://127.0.0.1:8848`) + 单页 Dashboard, **无需任何 API Key 即可体验核心链路** (离线降级模式, 且降级状态显式可见)。六大能力如下:

1. 技能库与技能契约 (资产的「仓库 + 规格书」)

- 51 个科研领域技能, 17 个领域**, 技能之间用 **62 条类型化关系边** (依赖 / 补充 / 前置等) 连接成图。
- 每个技能是一份 **Skill Contract**: 能力描述、适用时机、执行指令、陷阱提示 (pitfalls)、验证方式 (verification)、五维质量分 (5D quality)。契约即规格——下游 (人、Agent、其他框架) 按同一份规格消费。
- 技能库持久化 (原子写盘)、并发安全 (读写锁 + 索引原子替换), Demo 状态与正式库可隔离。

2. 三档检索路由 («找对技能» 的核心)

档位	机制	适用场景
bm25	关键词倒排, 纯本地	无网络 / 无模型, 兜底可用
hybrid	BM25 + 语义向量 + 领域/标签结构信号, 加权融合 (当前权重 0.70 / 0.20 / 0.10) + 多路一致性加成	常规生产路径
fabric	在 hybrid 之上叠加 关系图扩展 (沿 62 条关系边补入关联技能) + LLM 重排	复杂任务、依赖链长的场景

每条检索结果带**证据链**: 命中通道归因 (channel attribution)、结构得分、关系扩展理由、重排得分。API Key 缺失时 fabric 自动降级为「图扩展可用、LLM 重排不执行」, 响应中显式返回 `degraded: true`、`components: {...}`、`degraded_reason`, **不允许静默降级**。

3. 编排与执行（「用上技能」）

Orchestrator 根据检索结果与技能依赖关系排出执行顺序（DAG），Agent 携带选中技能的契约执行科研任务；执行轨迹、工具调用、产物均被记录，成本按 token 计入请求级账本。

3.5 汇报演示模式（对话式入口）

访问 `/static/briefing.html` 进入演示页：**输入任何科研问题**（不限领域、不限预设），系统以对话流形式逐段展示五阶段闭环——检索对比 → **LinUCB 策略选择**（候选池按历史反馈排序，展示预测收益 / 探索奖励 / 优先级）→ 依赖编排 → 技能驱动执行与盲评 → **反馈回流与蒸馏**（更新前后预测对比表 + 新候选技能卡），并附真实成本与 Token 记账。任意问题的回答都会展示「这次调用了哪些技能、旧数据如何影响本次排序、本次反馈如何改变下一次」。

4. 评估体系（「有没有效」——分主观与客观两条线）

- **主客观分离**：LLM judge 盲评（主观）与 **skill-derived reference coverage**（从技能契约的陷阱/验证要点中抽取参照点做覆盖判分，客观但需注明同源偏置）分开呈现，不混为一谈。
- **数据集治理**：任务集分为 **DEV（调参集）** 与 **HELD-OUT（冻结测试集）**，措辞与标注路径刻意不同源；Dashboard 与报告明确区分两套数字，最终结论只允许引用 held-out。
- **Provenance 可追溯**：`out/manifest.json` 统一记录 git commit、模型、温度、检索权重、各实验文件哈希；README / Dashboard / 报告的数字全部由 `config/experiment.yaml + scripts/build_report_data.py` 单一来源生成，杜绝手工同步漂移。

5. 选择策略与技能进化（「持续变好」）

- **Shared LinUCB 上下文老虎机**（v0.3 重写）：全局共享参数，上下文 $\phi(\text{task}, \text{skill})$ 编码任务词袋、技能词袋、领域 one-hot、交互与元特征，**跨进程哈希稳定**。实测一个技能的反馈可泛化影响同领域未评估技能（同领域技能的预测变化约为异领域的 1.24 倍）；预测奖励随轮次真实增长（0 → 0.52 → 0.73），探索奖励同步衰减。
- **技能进化**：从执行轨迹中提炼候选技能（parent、diff、质量、风险、相似度），经质量准入后回流技能库。
- **实时反馈回流（演示闭环内已接入）**：每次任务执行后，盲评奖励即时写回共享参数 θ ——**所有技能**的预测随之更新（同领域变化最大），排序随每一次任务演化；实测第二次相似请求中 5 个技能预测转为非零、探索奖励同步衰减，「根据旧数据迭代」全程可见、可追溯。
- **诚实的统计结论**：实验三采用 mean $\pm$ std 与 4000 次配对 bootstrap。当前样本下 LinUCB 与随机选择 **不可区分**（95% CI 跨 0），Demo 如实标注「本实验仅验证闭环连通性」，不包装成 RL 收益。

6. 跨框架导出与工程化（「不是孤岛」）

- **适配器导出**：技能库可一键导出为 `.claude/skills`、`.agents/skills`、`.cursor/skills`、`.gemini/skills` 及 `AGENTS.md`，供 Claude Code、Cursor、Gemini CLI 等直接加载。
- **工程化**：pytest 测试（含跨技能泛化、task conditioning、跨进程可复现性、结构护栏）、`verify.py` 31 项交付自检、前端渲染自检 5 项、token 鉴权、参数校验 422、稳定 CRC32 哈希。

二、实验证据：它到底有没有用（按证据等级如实呈现）

证据等级	含义	本项目对应结论
Demonstrated	有直接实验数据支撑	闭环链路可运行；学习曲线真实（预测奖励单调上升、探索衰减）；跨技能泛化在单元测试中被锁定
Supported	有数据但有条件/偏置	技能使盲评平均分 8.22 → 8.86（+7.8%）；skill-derived 覆盖率 81.3% → 92.8%（+11.5pp）；技能采纳率 67%–91%（均基于 dev 集，coverage 指标有同源偏置）
Preliminary	方向性信号，样本不足	LinUCB 与随机在当前样本下不可区分（差值 -0.037，95% CI [-0.089, 0.022] 跨 0），仅验证策略闭环
Not yet validated	尚未验证	held-out 集上的执行质量实验（exp2/exp3）；技能进化对长期任务成功率的影响；产物级（sandbox）评估

dev 与 held-out 的差异（本轮最重要的发现）

fabric 检索指标	DEV（调参集）	HELD-OUT（冻结测试集）	说明
gold 召回率	95.8%	86.1%	held-out 措辞与组合方式不同源（词面重合度仅 0.052）。编排完整度大幅下降说明 dev 高分含过拟合成分 ；关系图覆盖（62 条人工边）是当前最薄弱环节。
完整覆盖任务占比	90.0%	66.7%	
编排完整度	92.5%	25.0%	

这一落差正是「防泄漏方法论」的价值：如果只用 dev 集汇报，会把被调参污染的数字带进产品。

三、它有什么用：业务价值

Agent 产品的常见痛点	SkillNet 提供的解法
能力沉淀靠「写死在提示词里」，无法复用、无法版本化	技能资产化：Skill Contract 统一规格，入库、检索、导出全生命周期管理
技能多了以后 Agent「找不准」，检索只靠关键词	三档检索 + 关系图扩展 + 证据链归因，每一步可解释
说不清「技能到底带来多少收益」，汇报靠感觉	评估管线：主客观分离、dev/held-out 治理、统计检验、manifest 追溯
技能库只进不出、良莠不齐，越用越乱	质量分 + 候选评估 + （v0.4 规划中）候选审批/回滚治理，隔离 Demo 状态
绑定单一模型/框架，迁移成本高	OpenAI 兼容接口 + 跨框架导出（Claude Code / Cursor / Gemini 等）

一个经过数据验证的判断：技能的核心价值体现在**要点覆盖率**的提升（+11.5pp，dev 集），而非盲评平均分（+7.8%）——因为均分存在天花板效应。这决定了接入业务时应优先关注「Agent 是否照做了该做的事」，而不是只看评分。

四、如何接入真实业务产品

4.1 SkillNet 在整体架构中的位置



图 1 · SkillNet 作为独立服务层，业务 Agent 按需取用技能；技能资产库由专家与进化共同供给

4.2 接入方式 A: HTTP 服务（推荐，改动最小）

以独立进程部署 SkillNet，业务 Agent 通过 API 消费。当前可用端点：

端点	方法	用途	鉴权
/api/search	POST	三档检索 (bm25 / hybrid / fabric) ， 返回候选技能 + 证据链 + 降级状态	开放
/api/route	POST	检索 + 编排：返回带依赖顺序的执行方案 (DAG)	Token
/api/run	POST	携带技能执行任务，返回轨迹 / 评分 / 成本	Token
/api/evolve	POST	从轨迹提炼候选技能（供人工审批）	Token
/api/demo	POST	五阶段闭环演示：检索对比 → LinUCB 策略选择 → 编排 → 执行盲评 → 反馈写回与蒸馏；接受任意任务文本	Token
/api/adapters	POST	导出为 Claude Code / Cursor / Gemini 等框架的技能目录	Token
/api/skills, /api/graph, /api/stats, /api/health, /api/results	GET	技能列表 / 关系图 / 库状态 / 健康检查 / 实验结果	开放

鉴权方式：设置环境变量 `SKILLNET_TOKEN` 后，受保护端点要求请求头 `x-skillnet-token`。接入方最小改造：在 Agent 的「工具/技能选择」环节调用 `/api/search`，把返回的技能契约注入系统提示或工具描

述，即可获得检索路由能力。

4.3 接入方式 B：静态导出（零依赖）

调用 `/api/adapters` 一次性导出为各框架认可的技能目录格式，直接放进业务仓库：适合不想增加服务依赖、只需要「把专家知识带进 Agent」的场景。缺点是失去检索路由、评估与进化能力。

4.4 业务知识 → 技能资产的迁移方法

把业务中的专家经验整理为 Skill Contract，示例骨架：

```
name: order-refund-policy
domain: 客服运营
capability: 处理订单退款申请，判断是否符合退款条件并给出处理路径
when_to_use: 用户提出退款、退货、仅退款相关诉求时
instructions:
  - 先核对订单状态与退款政策矩阵（7 天无理由 / 质量问题 / 超期）
  - 质量问题需引导上传凭证，走极速退款通道
  - 超期订单需先申请售后期_special 审批
pitfalls:
  - 不要在未核对物流签收状态时承诺退款
  - 部分退款与整单退款的优惠券回退逻辑不同，勿混用
verification:
  - 回复中明确给出时效与到账路径
  - 涉及审批的必须告知审批入口
relations:
  # 类型化关系：依赖 / 前置 / 补充
  - {type: depends_on, target: order-status-query}
```

迁移建议：**从高频且易判错的任务开始**（判错代价大、规则可枚举的场景收益最高），每个领域先沉淀 3-5 个种子技能，把 SkillNet 的检索命中率和采纳率跑起来，再批量扩充。

4.5 分阶段接入路线

阶段	做什么	验收标准
Phase 1 只读接入（1-2 周）	部署服务；Agent 调 <code>/api/search</code> 取技能并注入提示词；接入 <code>/api/stats</code> 做埋点看板	检索采纳率有埋点；同任务对比「带技能 / 不带技能」的要点覆盖率与人工抽检质量
Phase 2 反馈闭环（2-4 周）	把业务侧人工反馈 / 工单结果回传为 reward，写入技能 stats；建立 DEV/HELD-OUT 业务任务集	能回答「哪些技能在拉高成功率」；检索权重按业务数据重调（只动 dev）
Phase 3 进化与治理（v0.4 后）	开启 <code>/api/evolve</code> 候选生成 + 人工审批门 + 回滚；高危技能加隔离与扫描	候选技能「生成 → 审批 → 上线 → 可回滚」全流程留痕，无人工审批不入库

4.6 安全与治理要求（供应链视角）

- 技能是**可执行知识**，会被下游 Agent 当指令加载——必须有人工审批门，生成技能默认标记 `untrusted` 并隔离（v0.4 规划）。
- 服务层启用 `SKILLNET_TOKEN`；对外仅暴露必要端点；路径校验已有，生成技能的指令扫描待补。
- 业务接入时建议为技能内容建立变更审核（diff + 审批留痕），防止提示注入内容借技能库扩散。

4.7 运行环境要求

启动方式	<code>python run.py</code> （需 Python ≥ 3.10 + 依赖 <code>fastapi/uvicorn/httpx/pydantic/numpy</code> ），默认端口 8848
离线模式	无 <code>DEEPSEEK_API_KEY</code> 时： <code>bm25</code> / <code>hybrid</code> / 图扩展 / Dashboard 全可用； <code>fabric</code> 的 LLM 重排显式降级并标注原因
在线模式	配置 DeepSeek 或任意 OpenAI 兼容端点后， <code>fabric</code> 重排、执行、评估、进化全链路可用，成本按 token 计量
自检	<code>python verify.py</code> （31 项）； <code>pytest tests/</code> ； <code>node tools/front_check.js</code> （5 项）

五、当前边界与已知限制（如实告知）

- 执行质量结论暂基于 dev 集**：`held-out` 的 `exp2/exp3` 尚未运行，技能对执行质量的提升属于偏乐观证据（Supported 级）。
- 编排完整度在 held-out 上仅 25%**：关系图覆盖是短板，接入业务前需要按业务任务补齐关系边。
- 选择策略收益未证实**：LinUCB 与随机当前不可区分，不应作为接入理由；接入理由应是「技能本身 + 检索 + 治理」。
- 评估仍以 LLM judge 为主**：产物级（sandbox 可执行判分）评估未接入，评估层级待升级。
- 生命周期状态机未完成**：`evolve` 产物仍直接入库，`candidate/approve/rollback` 为 v0.4 首要事项。
- 模型层单供应商绑定**：当前执行/评估默认 DeepSeek，抽象 `ModelProfile` 待做。

六、快速开始

```
# 1. 启动（本仓库已装好依赖）
python run.py                # 打开 http://127.0.0.1:8848 即是 Dashboard

# 2. 自检
python verify.py             # 31 项交付自检
python -m pytest tests/ -q    # 工程测试

# 3. 汇报演示页（对话式入口，推荐现场使用）
# 浏览器打开 http://127.0.0.1:8848/static/briefing.html
# 输入任意科研问题，逐段展示：检索对比 / LinUCB 策略选择 / 编排 / 执行盲评 / 反馈回流与蒸馏

# 4. API 方式调用同一闭环
curl -X POST http://127.0.0.1:8848/api/demo -H "Content-Type: application/json" \
  -d '{"task": "单细胞测序数据的质控、聚类与细胞亚群注释", "k": 5}'

# 4. 导出给 Claude Code / Cursor 等框架
curl -X POST http://127.0.0.1:8848/api/adapters
```

本文档中的全部数字来自 `out/*.json` 与 `out/manifest.json`（单一数据源生成），`dev` 与 `held-out` 结果分开存放；证据等级标注遵循 Demonstrated / Supported / Preliminary / Not yet validated 四级约定。